

OCP ACCESS TRACKING DASHBOARD

OpenShift Access Tracking & Reporting

Who can reach what, on every cluster: for the people who hold access, and for the quarterly review that must prove it.

Application 5.8.0 · chart 0.70.12 · every screenshot is from the lab on 2026-10-08: the running dashboard, or a report it generated

THE PROBLEM

Access on OpenShift fails silently

9

RoleBindings granting **admin**, **view** and **edit** to groups that have never existed, found on the reference cluster.

oc get rolebinding reports every one of them as healthy.

A binding names a group that does not exist

It looks healthy and grants nobody.

A binding names a person, not a group

Removing them from the directory revokes nothing: it survives offboarding.

A group silently stopped being refreshed

The sync still reports success.

WHY NOW

Two needs, one source of truth

Self-service

Everyone can answer “what access do I have, and how did I get it” without a ticket, an oc login with rights they do not have, or a platform engineer.

Quarterly access review

Auditors need evidence that names the access, the reviewer, the time and the outcome, and shows what changed since the last review. SOC 2, ISO 27001 A.5.18 (which replaced A.9.2.5 of the 2013 edition) and SOX centre on dormant, orphaned and privileged access.

The same observed data answers both: one view of access per person, and evidence for the review, drawn from it.

What it is

Read-only

It observes; it never creates or edits a GroupSync CR, a binding or a group.

Every cluster, one view

One dashboard on the host cluster reads every joined cluster, each with that cluster's own token.

Signed in with OpenShift

People log in the way they already do; the cluster decides what each person may see.

Evidence on demand and on schedule

Eleven reports as PDF/A, HTML, CSV and JSON, generated by hand or by a schedule.

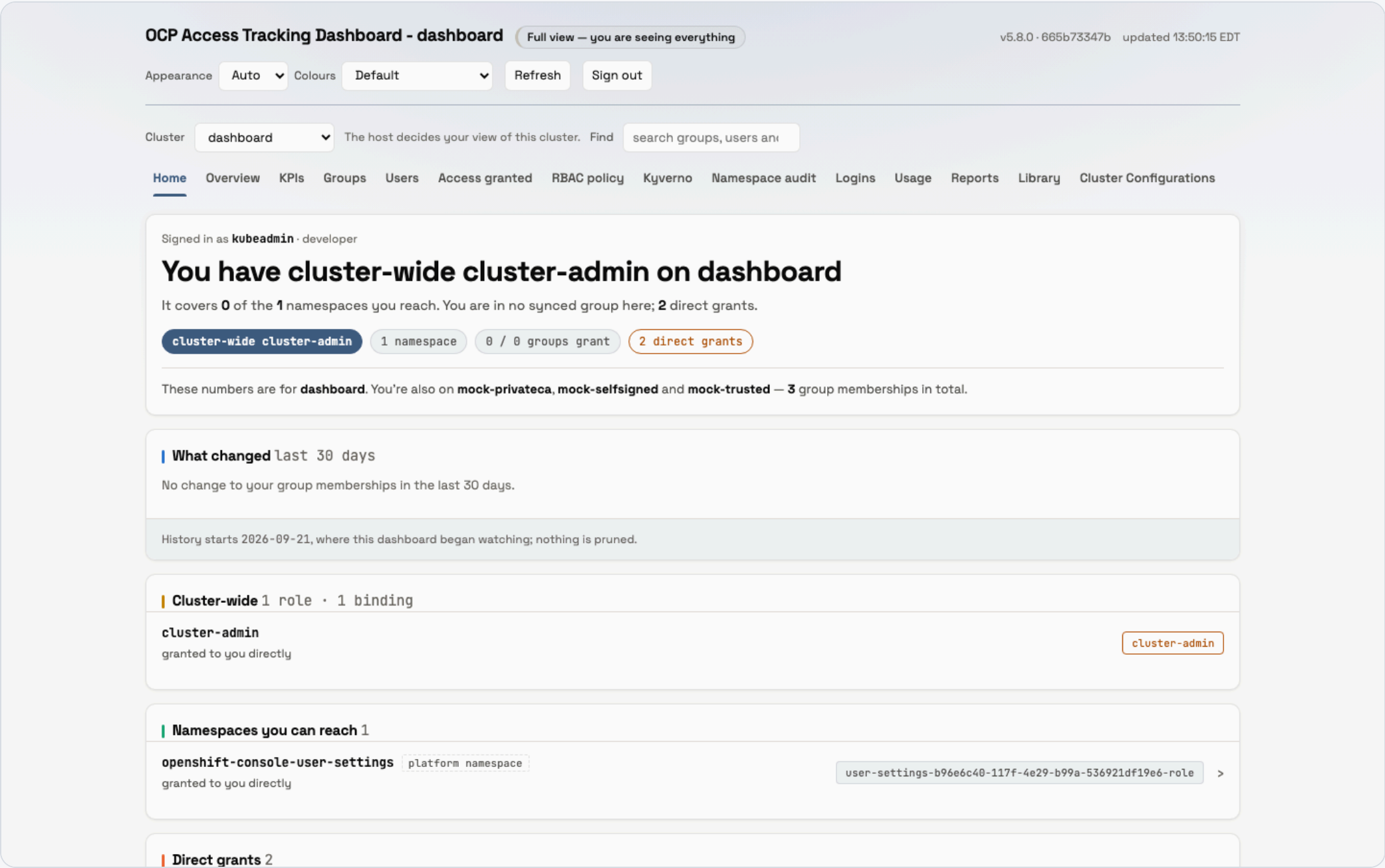
FOR EVERYONE

Self-service: everyone sees their own access

Home is always the reader's own access, administrators included.

It shows the cluster-wide roles, the namespaces they reach and how: through a group, or a grant naming them directly.

A view someone may not open shows a named refusal, never a blank page: an empty audit tab would read as a healthy cluster.



Screenshot: Home, light theme, the lab at 5.8.0 · ACCESS_CONTROL.md and the README for the refusal rule

For owners and auditors: what a group review cannot see

Grants that name a **person** rather than a group survive offboarding, and every group-based review misses them.

The page ranks namespaces by the worst privilege granted and names who is exposed.

Operational goal: zero rows. At zero, one directory change offboards a person from the entire cluster, and an access review that reads clean is clean.

OCP Access Tracking Dashboard - dashboard

Full view — you are seeing everything

v5.8.0 · 665b73347b updated 13:50:27 EDT

Appearance

Auto

Colours

Default

Refresh

Sign out

Cluster

dashboard

The host decides your view of this cluster.

Find namespace

filter by name or label

Export

CSV

JSON

12 rows

Home

Overview

KPIs

Groups

Users

Access granted

RBAC policy

Kyverno

Namespace audit

Logins

Usage

Reports

Library

Cluster Configurations

Namespace audit

High exposure

highest risk on this cluster

Namespaces at risk

5

People exposed

7

Grants to migrate

12

Critical

0

High

2

Why this is a risk

Offboarding does not revoke it

Removing someone from an LDAP group revokes their access everywhere at once. A binding that names the **person** keeps granting after they leave the team, change role, or leave the company — until somebody remembers it exists.

Invisible to access review

Every group-based review — including the rest of this dashboard — answers "who is in the group?". These grants are attached to no group, so a clean review can coexist with standing access nobody approved.

No approval trail

Group membership is granted in the enterprise directory, with a request and an approver. A hand-made binding records only who ran oc, and only for as long as the audit log is retained.

Cluster-wide direct grants · 2

These name a person at the **cluster** scope, so each one grants every namespace on this cluster — all 128 of them. A scope, not a namespace: the table below ranks namespaces only, and the **Namespaces at risk** tile above counts it as one entry.

RISK	HIGHEST PRIVILEGE	PEOPLE	GRANTS	WHO IS EXPOSED
High	edit	2	2	dana.lee · jdoe

Each binding, with the name to delete, is in **Every grant** below: choose *cluster-wide* in its Namespace selector.

Exposure by namespace

Ranked by the **worst privilege granted**, not by count — one forgotten cluster-admin matters more than twenty view grants.

RISK ▼	NAMESPACE	HIGHEST PRIVILEGE	PEOPLE	GRANTS	WHO IS EXPOSED
--------	-----------	-------------------	--------	--------	----------------

Screenshot: Namespace audit, light theme, the lab at 5.8.0 · the page's own text

EVIDENCE

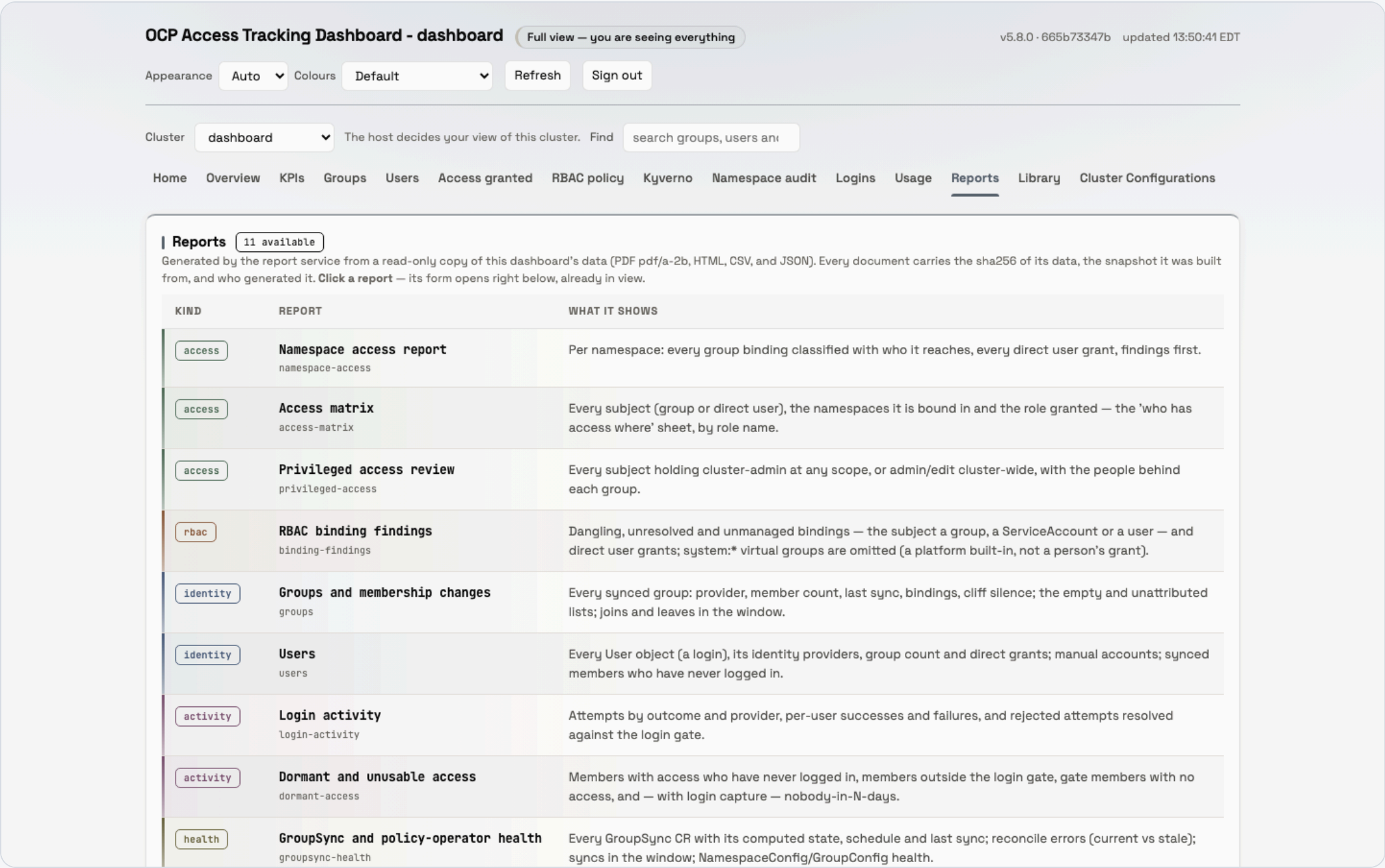
Eleven reports, one per review question

Who can reach each namespace; the access matrix; privileged access; binding findings.

Groups and their changes; users; login activity; dormant access; sync health.

A one-page compliance snapshot, and an access certification pack for sign-off.

Every document carries the sha256 of its data, the snapshot it was built from, and who generated it.



Screenshot: Reports, light theme, the lab at 5.8.0 · docs/reports/README.md

The certification pack: approve or revoke, line by line

A campaign with a due date and a named reviewer.

Every group's bindings, and every grant naming a person directly, with empty Approve, Revoke and Comment columns, and the members behind each group.

“This pack records the state the dashboard observed; it does not change anything.” The reviewer signs the last page.

Campaign

Campaign	E2E walk 2026-10-08
Due	2026-11-07
Reviewer	kubeadmin
Scope	all
Cluster	dashboard

For each line: tick Approve to keep the access as it stands, Revoke to remove it, and write the reason in Comment. Sign the last page. This pack records the state the dashboard observed; it does not change anything.

Group: app-ocp-rbac-alpha-cluster-admin

Bindings of app-ocp-rbac-alpha-cluster-admin

namespace	role	binding	classification	Approve	Revoke	Comment
(cluster-scoped)	admin	app-ocp-rbac-alpha-cluster-admin-crb	ok	☐	☐	

Members of app-ocp-rbac-alpha-cluster-admin

member	name	logged in	first seen
jane.smith	Jane Smith	yes	2026-09-21T06:15:27Z

Group: app-ocp-rbac-alpha-cluster-audit

Bindings of app-ocp-rbac-alpha-cluster-audit

namespace	role	binding	classification	Approve	Revoke	Comment
(cluster-scoped)	view	app-ocp-rbac-alpha-cluster-audit-crb	ok	☐	☐	

Members of app-ocp-rbac-alpha-cluster-audit

member	name	logged in	first seen
bob.wilson		unknown	2026-09-21T06:15:27Z

Screenshot: the access-certification report as HTML, generated on the lab at 5.8.0

PROVENANCE

Evidence an auditor can trust

The sha256 of the report's data, the snapshot it was read from, and its schema.

Who generated it (a person verified by the OpenShift login, or the schedule by name) and when.

PDF/A-2b for archiving; a report diff answers “what changed since the last review”.

Login attempts are kept 400 days, so a year-over-year review still sees the previous cycle.

Screenshot: the report's provenance, the lab at 5.8.0 · values.yaml (login retention) · SPEC_F3 (report diff)

HANDLING: INTERNAL — ACCESS REVIEW EVIDENCE

Access certification pack

OCP Access Tracking Dashboard · cluster **dashboard** · generated 2026-10-08T17:51:55Z by kubeadmin (proxy-verified, ticket from the dashboard)

sha256 of the report data: 47663db7c718920af1319df79c4b2a6ed5fe56e0315293ae2e40d058826259f8

Print / Save as PDF

Provenance and coverage

Provenance

Handling	Handling: internal — access review evidence
Cluster	dashboard — https://kubernetes.default.svc
Generated at (UTC)	2026-10-08T17:51:55Z
Generated by	kubeadmin (proxy-verified, ticket from the dashboard)
Run id	20261008T175155.077978Z-d180
Report service	5.8.0 @ 665b73347b
Data as of	snapshot 2026-10-08T17:51:46.627061Z (taken 0 min before generation; bindings refresh every 3600 s); schema 20
Last poll	2026-10-08T17:50:46Z — ok
Namespaces	every namespace on the cluster was read — a namespace with no grants is reported as having none
User objects	read — 'logged in' means a User object with an identity exists
Login capture	on — attempts are recorded; see the note for since when
History retained since	membership_event: 2026-09-21T06:15:27Z, sync_event: 2026-09-21T06:15:27Z, login_event: 2026-09-21T06:15:28Z
Includes membership rosters	yes
Parameters	campaign=E2E walk 2026-10-08, due=2026-11-07, group_mnemonic=[], groups=[], include_members=True, reviewer=kubeadmin, users=[]
PDF	variant pdf/a-2b; font DejaVu Sans 2.37

Every namespace on the cluster was read, so a requested namespace that does not exist is reported as such, and a namespace with no observed binding is reported as having no grants.

User objects were read; 'logged in' means a User with an identity exists.

Login attempts are recorded since 2026-09-21T06:15:28Z (last read 2026-10-08T17:50:46Z); nothing before that was ever observed.

Scope: direct bindings only; role rules are not evaluated — this is not an effective-permissions calculation.

EVERY QUARTER

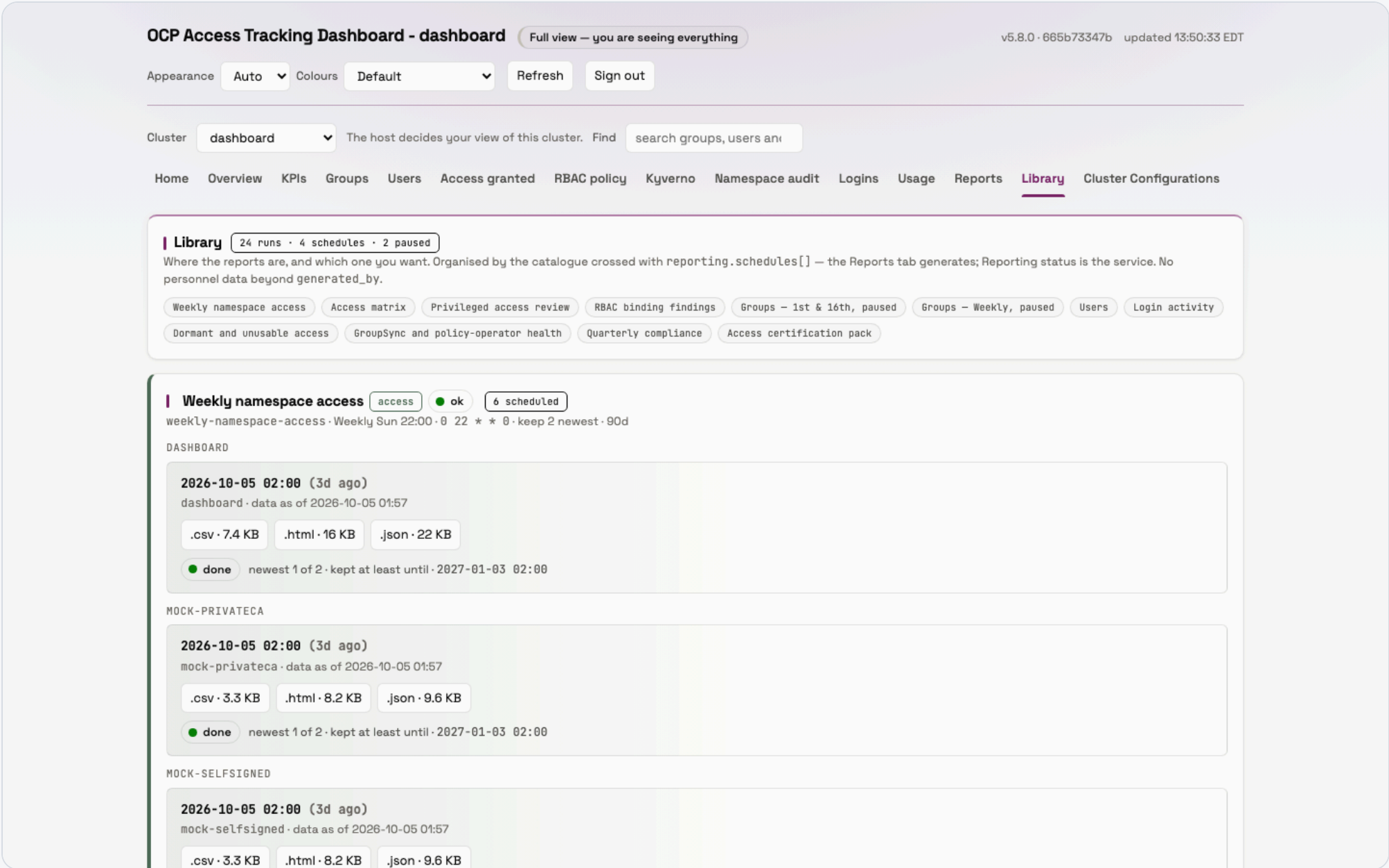
Scheduled, not remembered

quarterly-compliance

0 5 1 1,4,7,10 *

05:00 New York time on 1 January, April, July and October; every run kept 2,555 days, about seven years, and the newest 12 per cluster beyond that.

Each scheduled run can send one CloudEvent to a webhook, and an alert fires when a schedule runs late.



Screenshot: Library, light theme, the lab at 5.8.0 · environments/crc.yaml (the quarterly schedule)

PART 2

How it works

Read-only by design: it reads clusters, and it never changes anyone's access.

Who sees what: the cluster decides

Tier	Who has it	What they see
Self	Everyone who can sign in	Their own access: Home, and their own rows
Auditor	Can list clusterrolebindings, for example through an auditor group	Every cluster-wide audit view, Reports and Library; Usage stays their own
Usage	Passes the usage question; by default the same one as Cluster-admin	The Usage tab: everyone's activity
Cluster-admin	Can update clusterrolebindings	KPIs and Cluster Configurations, plus all of the above

A SubjectAccessReview on the cluster decides each tier; the answer is cached 60 s, and any failure falls back to Self.

Architecture at a glance

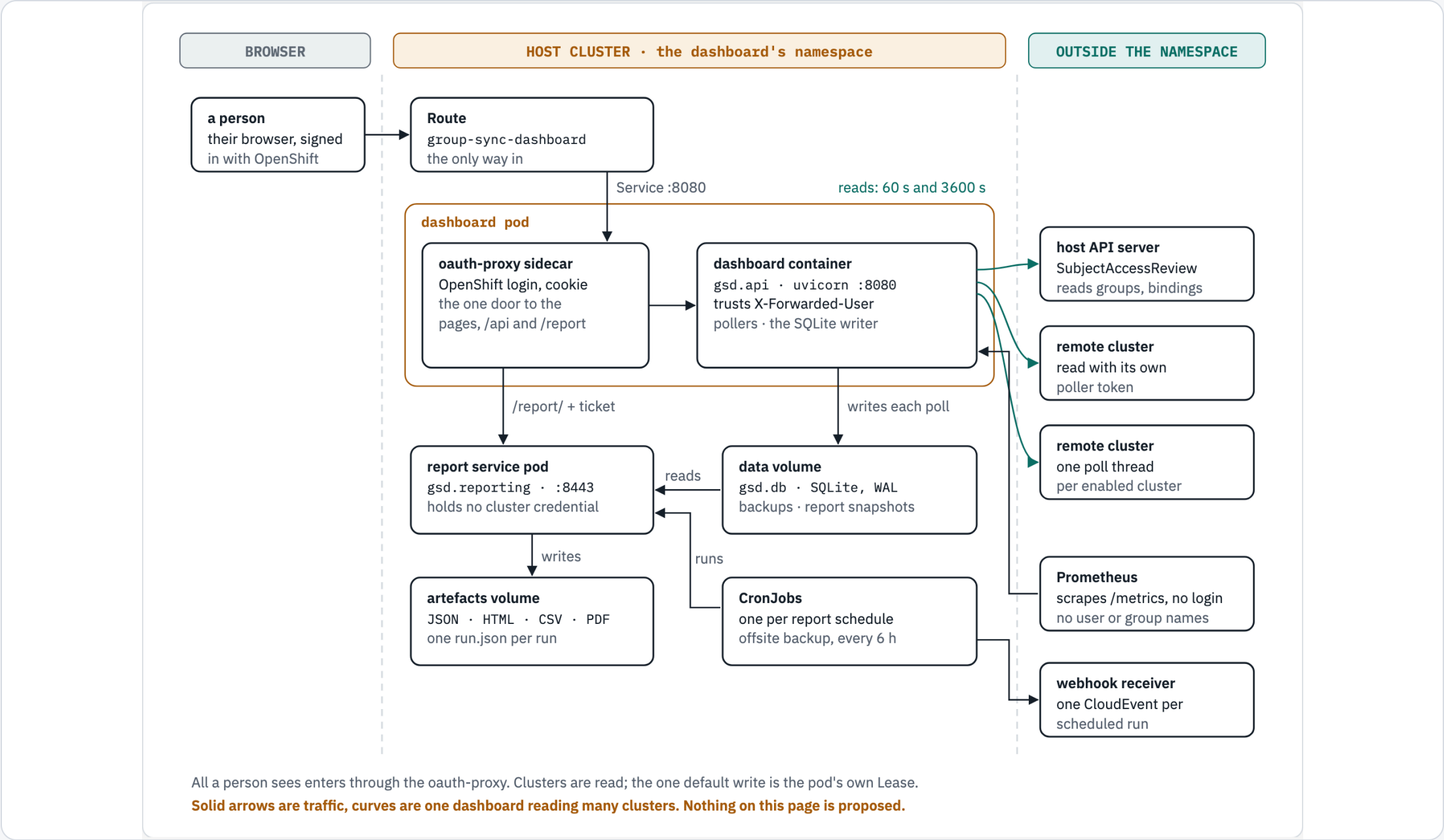


Figure 1 of docs/guides/architecture-overview.md, drawn from the code at 5.8.0

Inside one poll cycle

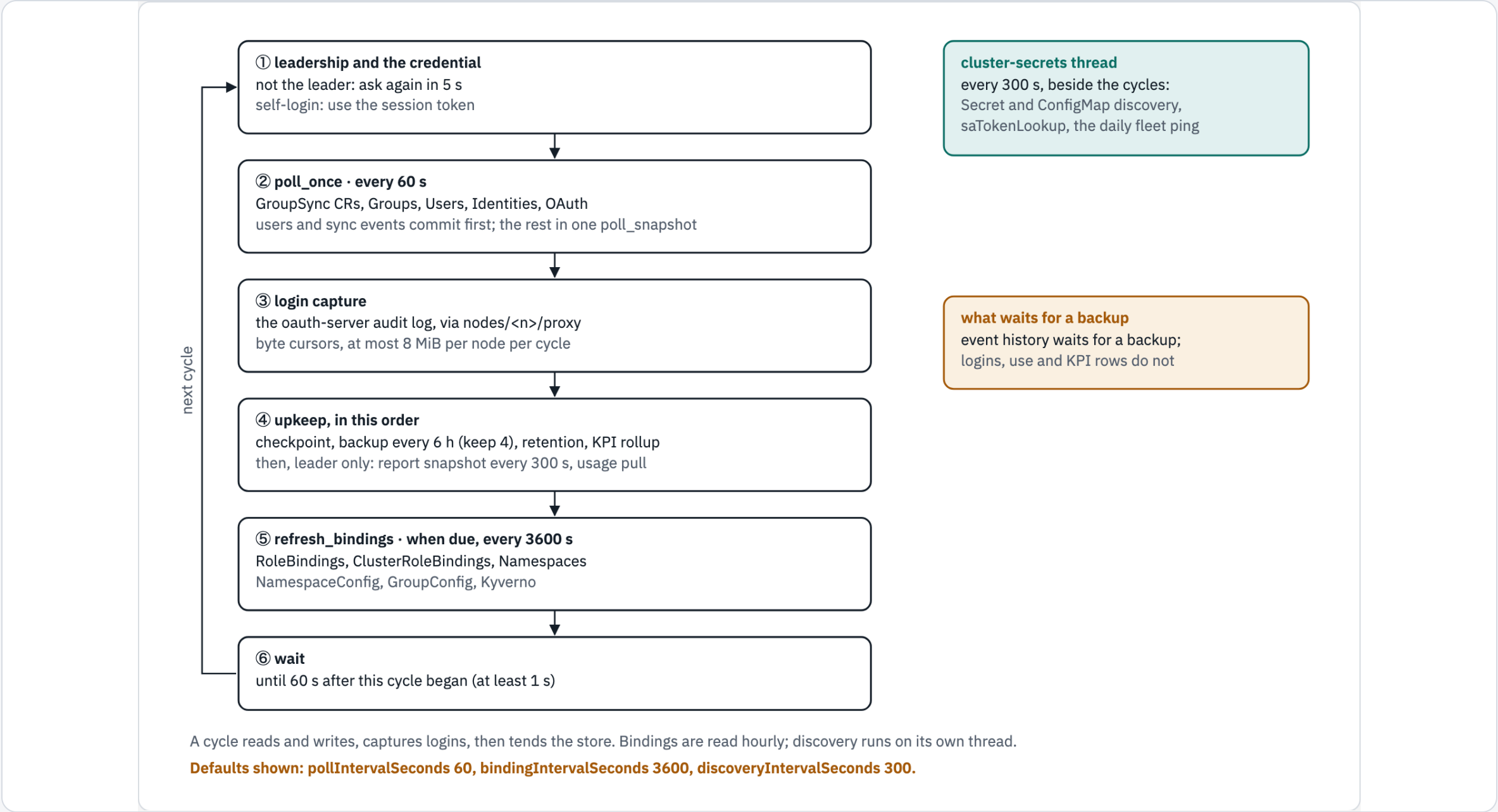


Figure 3 of docs/guides/architecture-overview.md

How a report is made

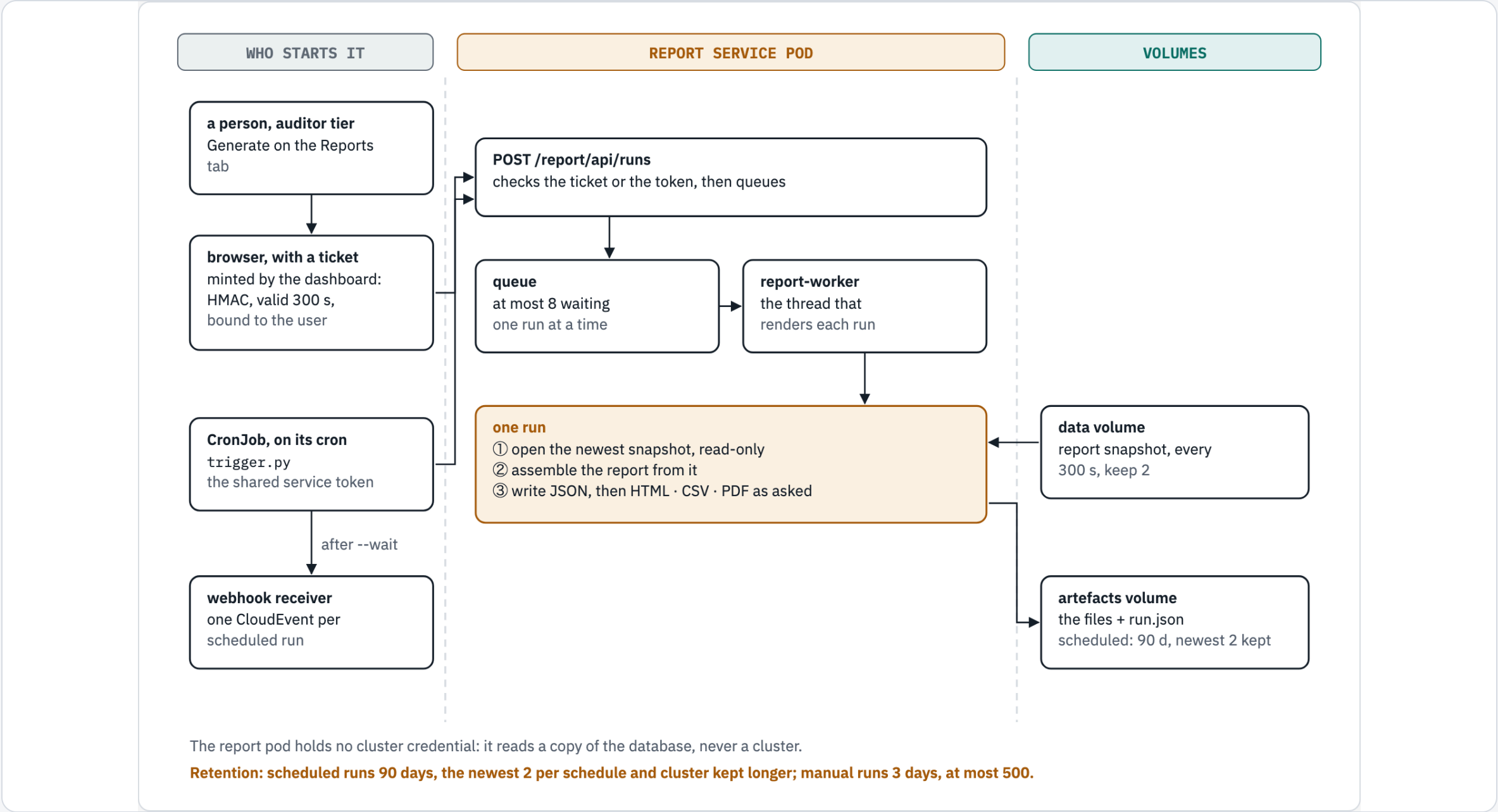


Figure 5 of docs/guides/architecture-overview.md

Credentials and trust boundaries

HOW	WHAT STARTS IT	WHAT CROSSES TO THE REMOTE	WHAT IS KEPT
steady state	every poll of every cluster	the remote's own poller token: reads, plus the remote-sar question	Secret gsd-cluster-<n>, re-read every 300 s
saTokenLookup	the discovery thread, leader only, when the writes switch is on	the fleet password, once: log in, read one Secret by name, revoke	writes the host Secret gsd-cluster-<n>
Rejoin	a cluster-admin presses Rejoin and types their own password	one login, a self-check (SSAR), one Secret read, revoke	the same Secret; the password is never kept

The browser never receives a cluster token: /api names the credential's kind only, and the report pod sees a 300 s ticket.

Polling only reads. A remote only ever gets a SubjectAccessReview, a SelfSubjectAccessReview and a login token, which is revoked.

Figure 6 of docs/guides/architecture-overview.md

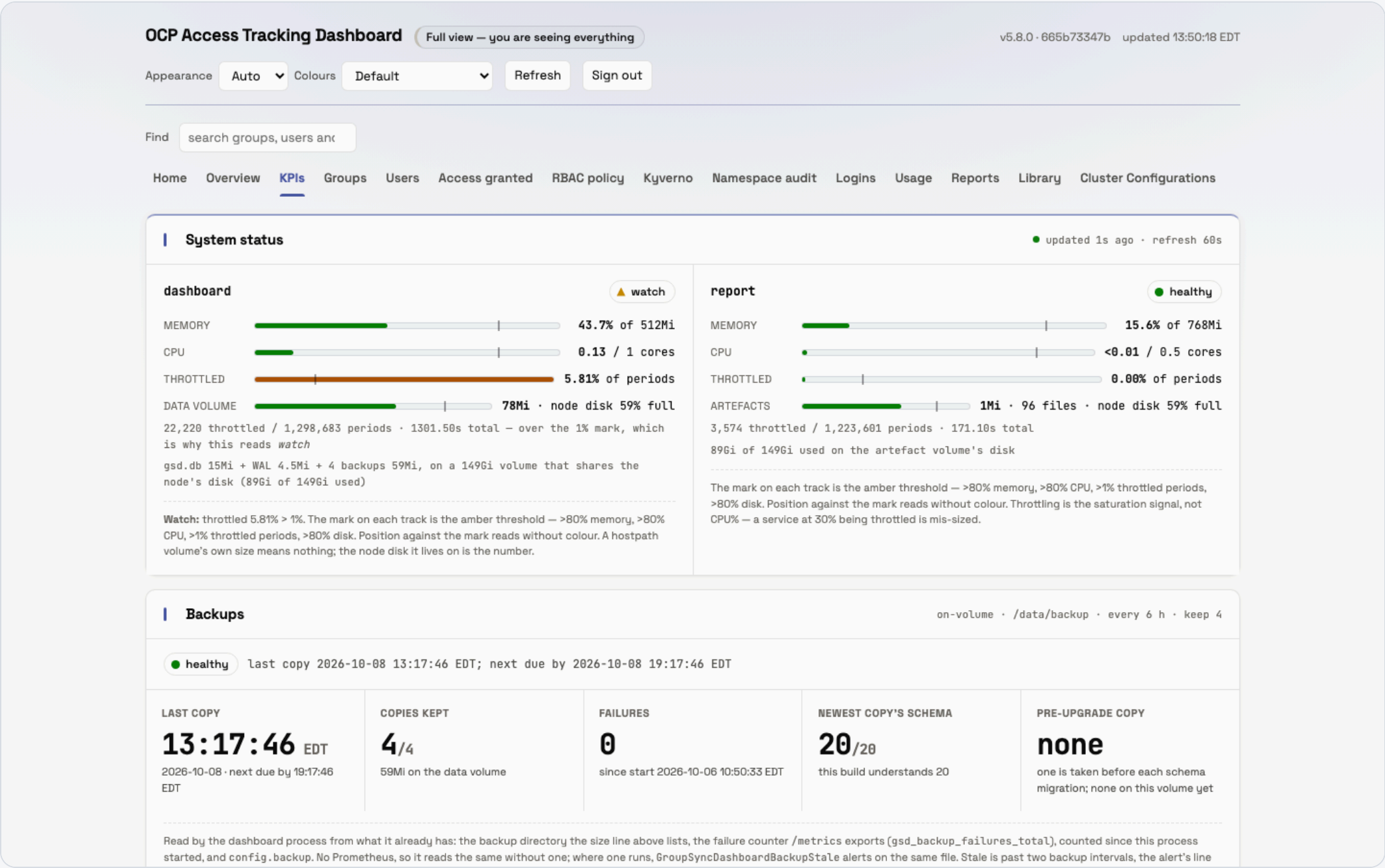
Operations: what the platform team runs

One Helm chart, ready for Argo CD; one dashboard pod and one report pod.

A database backup every 6 h with 4 kept, and an offsite copy every 6 h.

Twenty Prometheus alerts, from a cluster that stops answering to a backup or a schedule running late.

The KPIs page shows the system's own health: memory, CPU, throttling, disk, backups.



Screenshot: KPIs, light theme, the lab at 5.8.0 · the chart's PrometheusRule and backup settings

NEXT STEPS

Rolling it out

1

Install

the chart on the host cluster, with Helm or Argo CD.

2

Join clusters

each with its own token: a Secret, saTokenLookup or Rejoin.

3

Name the auditors

the groups that review access everywhere.

4

Schedule the evidence

a quarterly compliance schedule, and a webhook for delivery.

No access changes on any cluster: the dashboard reads, and the reports record what it read.

Its only writes are its own: a leader Lease on the host and, when a cluster joins by saTokenLookup or Rejoin, that cluster's token Secret on the host and a login on the remote that it revokes.

THANK YOU

Questions, and where to read more

`architecture-overview.md`

the architecture in seven figures

`reference-architecture.md`

the full reasoning, claim by claim

`ACCESS_CONTROL.md`

who sees what, and why

`docs/reports/README.md`

the eleven reports and their schedules

All documents are in the group-sync-dashboard repository, under docs/